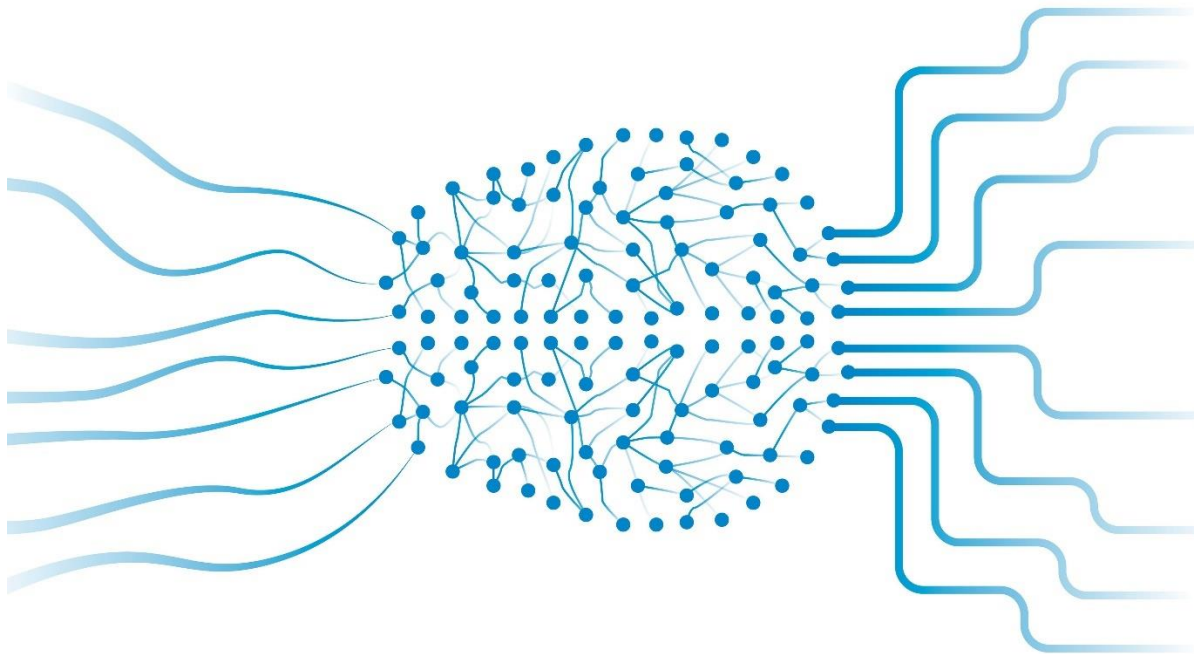




Team 9 Members:

Name :	ID :
Amr Adel Abdelalim	2000959
Abdelrahman Yasser Adel	2000414
Mariam Mahmoud Adel	2100264

Submitted to: ENG. Abdallah Awadallah.

Under the Supervision of: DR. Hossam Hassan.



Table of Contents :

Introduction:	3
Phase 1:	4
Why XOR?.....	4
Why We Need a Hidden Layer?	5
Why Use Tanh in the Hidden Layer?	5
Why Use Sigmoid in the Output Layer?	6
Mini Manual Example: How a Hidden Layer Solves XOR	6
Library Architecture:	8
Layers.py - Foundation Layer Components :	8
activations.py - Non-Linear Activation Functions :	10
losses.py - Loss Function Implementation :	13
optimizer.py - Stochastic Gradient Descent :	13
network.py - Sequential Model Architecture :	14
Gradient Checking :	15
XOR Neural Network Experiment :	16
Results:	17
Conclusion:	18



Introduction:

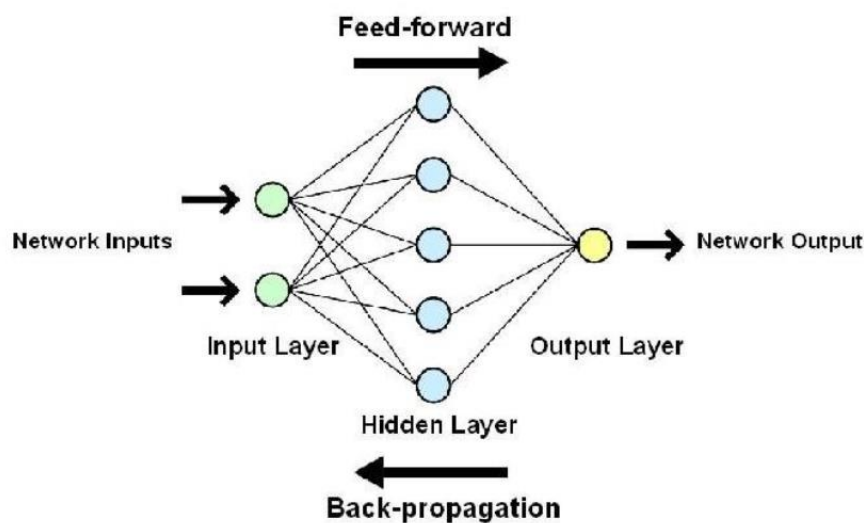
The purpose of this project is to design and implement a modular neural network library using only Python and NumPy, without relying on high-level machine learning frameworks. By constructing each component manually: dense layers, activation functions, optimization algorithms, and backpropagation.

The project provides an in-depth understanding of the mathematical and computational mechanisms that enable neural networks to learn.

The first phase focuses on building and validating the core library. Gradient checking is used to verify the correctness of the analytical derivatives, while the XOR problem serves as a benchmark to confirm that the implemented forward and backward propagation yield correct learning behavior.

In the second phase, the library is extended to train an autoencoder for MNIST image reconstruction. The encoder's latent space is then utilized for downstream classification using an SVM model. This allows us to examine the effectiveness of the learned representations and compare the pipeline's performance against TensorFlow implementations.

By the end of the project, we achieve not only a functioning deep learning library, but also a complete workflow that includes unsupervised learning, feature extraction, supervised classification, and baseline comparison with industry-standard tools.





Phase 1:

Why XOR?

XOR is considered the "Hello World" of neural networks because:

1. **It is very simple**

Only four input–output pairs:

Input	Target
0 0	0
0 1	1
1 0	1
1 1	0

2. **It is NOT linearly separable**

A straight line cannot separate the outputs of XOR.

This means [a simple linear model cannot solve it](#).

3. **It requires a hidden layer + non-linear activation**

XOR forces the neural network to use:

- Dense layers
- Activation functions
- Backpropagation
- Learned weights

4. **Perfect validation for a custom neural network library**

If your library can solve XOR → we know the system works.

If the network successfully learns XOR, it proves that:

- Forward and Backward passes are correct
- Gradients are correct
- Weight updates work
- Activation functions work
- The library can learn non-linear patterns



Why We Need a Hidden Layer?

The hidden layer allows the network to create two separate linear boundaries. When combined, these boundaries form a **non-linear** shape that can solve XOR.

Example architecture:

Input (2)

→ Dense(2 → 4)

→ Tanh

→ Dense(4 → 1)

→ Sigmoid

Why Use Tanh in the Hidden Layer?

The Tanh (hyperbolic tangent) function takes any real value and maps it to the range :

-1 to +1.

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

This helps because XOR naturally has:

- positive patterns
- negative patterns
- symmetry around zero

Example:

$\tanh(0.5) \approx 0.462$

$\tanh(-0.5) \approx -0.462$

This positivity/negativity allows the hidden neurons to “flip” or “invert” patterns—something linear layers cannot do.



Why Use Sigmoid in the Output Layer?

The sigmoid function maps numbers to the range **0 to 1**.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

XOR output is either **0 or 1**.

Sigmoid maps any value to the range **(0, 1)**:

$\text{sigmoid}(0.8) \approx 0.69$

$\text{sigmoid}(3.5) \approx 0.97$

$\text{sigmoid}(-2) \approx 0.12$

So sigmoid naturally works for binary classification.

Mini Manual Example: How a Hidden Layer Solves XOR

simple numerical example to show **why a hidden layer works**.

the input :

$x = (1, 0)$

target = 1

Assume hidden layer weights start like this:

Hidden Neuron 1: $w_{11} = 1, w_{21} = 1$

Hidden Neuron 2: $w_{12} = 1, w_{22} = -1$

Step 1 — Compute Hidden Layer Inputs

For input (1,0):

$h1_input = (1)(1) + (0)(1) = 1$

$h2_input = (1)(1) + (0)(-1) = 1$

Step 2 — Apply Activation (tanh)

$h1_output = \tanh(1) \approx 0.761$

$h2_output = \tanh(1) \approx 0.761$



These two hidden neurons are “detecting patterns” in the input.

Step 3 — Output Layer

Assume output weights:

$$v1 = 0.8$$

$$v2 = -0.7$$

$$\text{bias} = 0.1$$

Compute output:

$$O_input = (0.761)(0.8) + (0.761)(-0.7) + 0.1$$

$$\approx 0.608 - 0.532 + 0.1$$

$$\approx 0.176$$

Apply sigmoid:

$$\text{sigmoid}(0.176) \approx 0.543$$

The output ≈ 0.54

The target = 1

So there is an **error**, and **backpropagation** adjusts the weights.

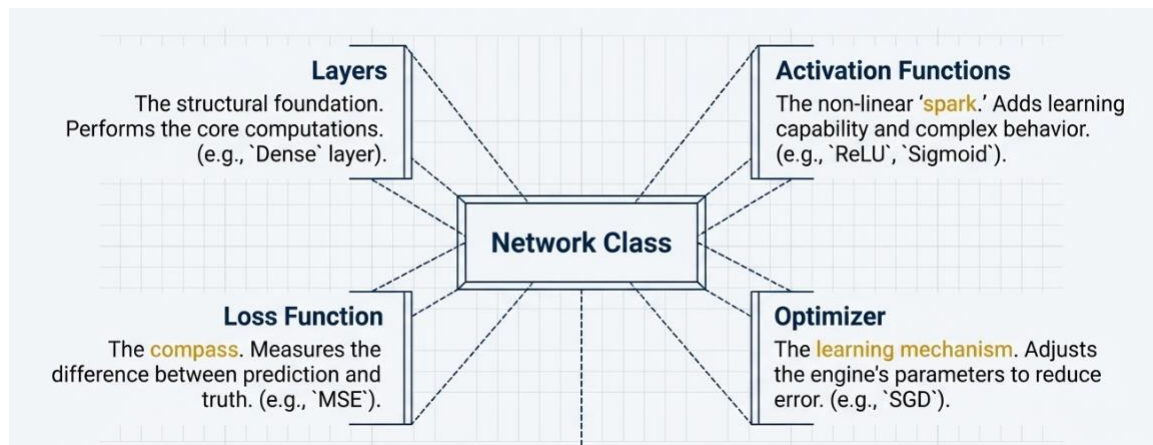
After enough updates, the network will produce a value close to 1.



Library Architecture:

lib/

```
init__.py    # Package initialization and exports
layers.py    # Base Layer class and Dense layer
activations.py # Activation functions (ReLU, Sigmoid, Tanh, Softmax)
losses.py    # Loss functions (MSE)
optimizer.py # Optimization algorithms (SGD)
network.py   # Sequential model for training
```



Layers.py - Foundation Layer Components :

The layers.py module provides the foundational structure for all neural network layers. It implements:

- An abstract base class Layer that defines the interface all layers must follow
- A concrete Dense (fully connected) layer implementation



class Layer(ABC):

Description:

Abstract base class for all neural network layers. Every layer must implement forward and backward propagation methods. This ensures a consistent interface across different layer types.

Attributes:

- input : stores the input passed to the layer during forward propagation.
- output : stores the output computed by the layer.

Methods:

- forward(input) :

Abstract method for computing the layer's output given an input. Must be implemented by subclasses.

- backward(output_gradient, learning_rate=None) :

Abstract method for computing gradients for backpropagation. Returns the gradient of the loss with respect to the layer's input. Must be implemented by subclasses.

class Dense(Layer)

Description:

Represents a fully connected (dense) layer. Computes the linear transformation:

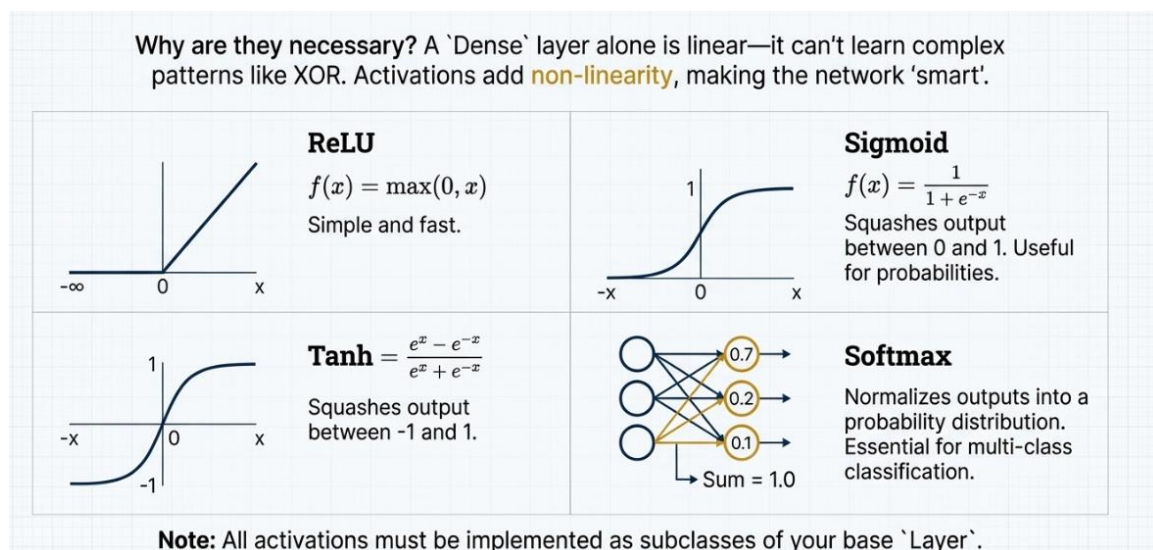
$$\text{output} = \text{input} \cdot \text{weights} + \text{bias}$$

Attributes:

- weights : weight matrix of shape (input_size, output_size), initialized using He initialization.
- bias : bias vector of shape (1, output_size), initialized to zeros.
- weights_gradient : stores the gradient of the loss with respect to weights.
- bias_gradient : stores the gradient of the loss with respect to bias.

**Methods:**

- **forward(input) :**
computes the output of the dense layer using the linear transformation. Stores the input and output for use in backpropagation. Returns the computed output.
- **backward(output_gradient, learning_rate=None) :**
computes the gradients of the weights, bias, and input using the chain rule:
 - $\text{weights_gradient} = \text{input.T} @ \text{output_gradient}$
 - $\text{bias_gradient} = \text{sum}(\text{output_gradient})$
 - $\text{input_gradient} = \text{output_gradient} @ \text{weights.T}$
Returns input_gradient for propagation to previous layers.
- **get_params() :** returns the list of trainable parameters [weights, bias].
- **get_gradients() :** returns the list of parameter gradients [weights_gradient, bias_gradient].

activations.py - Non-Linear Activation Functions :



class ReLU(Layer)

Description:

Rectified Linear Unit activation function. Sets negative inputs to 0. Commonly used in hidden layers to avoid vanishing gradients.

Attributes:

- input — stores the input passed to the layer.
- output — stores the computed output.

Methods:

- forward(input) : computes $\text{output} = \max(0, \text{input})$ and stores input/output.
 - backward(output_gradient, learning_rate=None) : computes gradient: $\text{output_gradient} * (\text{input} > 0)$.
-

class Sigmoid(Layer)

Description:

Sigmoid (logistic) activation. Squashes input into the range (0, 1). Often used in output layers for binary classification.

Attributes:

- input — stores the input.
- output — stores the computed output.

Methods:

- forward(input) : computes $\text{output} = 1 / (1 + \exp(-\text{input}))$, with clipping to avoid overflow.
 - backward(output_gradient, learning_rate=None) : computes gradient: $\text{output_gradient} * \text{output} * (1 - \text{output})$.
-



class Tanh(Layer)

Description:

Hyperbolic tangent activation. Squashes input into the range (-1, 1).

Attributes:

- input — stores the input.
- output — stores the computed output.

Methods:

- forward(input) : computes $\text{output} = \tanh(\text{input})$.
 - backward(output_gradient, learning_rate=None) : computes gradient: $\text{output_gradient} * (1 - \text{output}^2)$.
-

class Softmax(Layer)

Description:

Softmax activation. Converts raw scores (logits) into probabilities. Outputs are in (0, 1) and sum to 1. Used for multi-class classification.

Attributes:

- input — stores the input.
- output — stores the computed probability vector.

Methods:

- forward(input) : computes $\text{output}_i = \exp(\text{input}_i) / \sum(\exp(\text{inputs}))$, with max-subtraction for numerical stability.
 - backward(output_gradient, learning_rate=None) : returns output_gradient (simplified, works well with MSE).
-



losses.py - Loss Function Implementation :

class MSE

Description:

Mean Squared Error (MSE) loss. Measures the average squared difference between predicted values and true values. Commonly used for regression tasks.

Attributes:

- None (all methods are static).

Methods:

- `loss(y_true, y_pred)` — computes the MSE loss:

$$L = \frac{1}{N} \sum (y_{\text{true}} - y_{\text{pred}})^2$$

Returns a scalar value representing the average error.

- `gradient(y_true, y_pred)` — computes the derivative of the loss with respect to predictions:

$$\frac{dL}{dy_{\text{pred}}} = \frac{2}{N} (y_{\text{pred}} - y_{\text{true}})$$

Returns an array of gradients for backpropagation.

optimizer.py - Stochastic Gradient Descent :

Learning Rate Selection:

The learning rate η is the most critical hyperparameter:

Learning Rate	Effect	Problem
Too Small ($\eta \ll 0.01$)	Slow convergence	Training takes very long
Optimal ($\eta \approx 0.01-0.1$)	Smooth convergence	Reaches minimum efficiently
Too Large ($\eta \gg 0.1$)	Oscillation/Divergence	Loss increases or oscillates



class SGD

Description:

Stochastic Gradient Descent (SGD) optimizer. Updates model parameters using gradients computed during backpropagation:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \cdot \nabla L(\theta)$$

Where:

- θ = parameters (weights, biases)
- η = learning rate
- $\nabla L(\theta)$ = gradient of loss w.r.t parameters

Attributes:

- learning_rate : step size used to update parameters.

Methods:

- update(layers) : iterates through all layers, and if a layer has weights and bias, updates them using:
 - weights = learning_rate * weights_gradient
 - bias = learning_rate * bias_gradient
- set_learning_rate(learning_rate) : sets a new learning rate for the optimizer.

network.py - Sequential Model Architecture :

The Sequential class provides a high-level interface for building, training, and evaluating neural networks. It orchestrates the interaction between layers, loss functions, and optimizers.

class Sequential

Description:

Sequential model for stacking layers in order. Handles forward and backward propagation, loss computation, and parameter updates. Provides training via fit and predictions via predict.



Attributes:

- layers : list of added layers.
- loss_function : loss function object (e.g., MSE).
- optimizer : optimizer object (e.g., SGD).

Methods:

- add(layer) : adds a layer to the model.
- compile(loss, optimizer) : sets the loss function and optimizer for training.
- forward(X) : performs a forward pass through all layers, returning output.
- backward(loss_gradient) : performs a backward pass through all layers to compute gradients.
- train_step(X, y) : executes a single training step: forward pass, loss computation, backward pass, and parameter update. Returns the loss for the step.
- fit(X, y, epochs, batch_size=None, verbose=True) : trains the model over multiple epochs. Supports mini-batch training and returns a history dictionary with epoch losses.
- predict(X) : computes outputs for given inputs using a forward pass.

Gradient Checking :

Description:

Verifies the correctness of backpropagation gradients in layers with trainable parameters by comparing analytical gradients (backward()) with numerical gradients computed via small perturbations:

$$\frac{\partial L}{\partial w_{ij}} \approx \frac{f(w_{ij} + \epsilon) - f(w_{ij} - \epsilon)}{2\epsilon}$$



Purpose:

- Detects errors in gradient computation.
- Ensures proper weight updates during training.

Method:

1. Forward pass on input X.
2. Backward pass to compute analytical gradients.
3. Compute numerical gradients using finite differences.
4. Calculate relative difference:

$$\text{difference} = \frac{\| \text{analytical} - \text{numerical} \|}{\| \text{analytical} \| + \| \text{numerical} \|}$$

5. If difference < 1e-5, gradients are correct.

Output:

- Relative difference and pass/fail message.
 - Sample values of analytical vs. numerical gradients.
-

XOR Neural Network Experiment :

Network Architecture:

- **Input Layer:** 2 neurons (for XOR inputs)
- **Hidden Layer:** 4 neurons, Tanh activation
- **Output Layer:** 1 neuron, Sigmoid activation
- **Optimizer:** Stochastic Gradient Descent (SGD), learning rate = 0.1
- **Loss Function:** Mean Squared Error (MSE)

Description:

A small feedforward neural network was built to learn the XOR function, which is a classic non-linearly separable problem. The network was implemented using a sequential stacking of layers (Dense + activation).

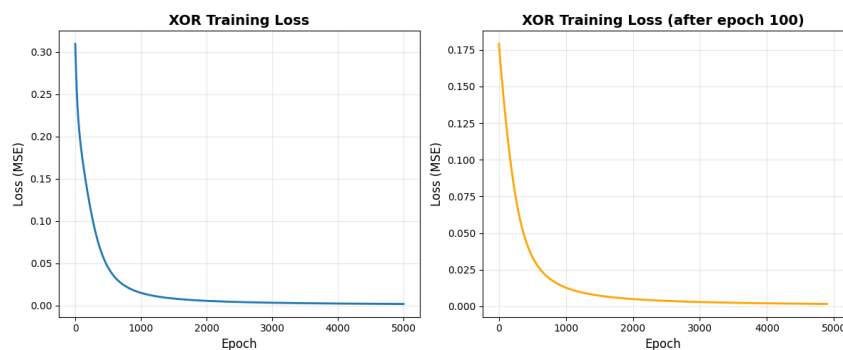


Training Procedure:

1. Model compiled with MSE loss and SGD optimizer.
2. Trained on the XOR dataset for 5000 epochs.
3. Loss monitored over training to evaluate convergence.

Results:

- The training curve shows a steady decrease in loss.
- The network successfully learned the XOR mapping, outputting values close to the expected targets 0 or 1.



FINAL RESULTS

Predictions:

Input	Prediction	Target
-------	------------	--------

[0. 0.]	0.0205	0
---------	--------	---

[0. 1.]	0.9571	1
---------	--------	---

[1. 0.]	0.9581	1
---------	--------	---

[1. 1.]	0.0505	0
---------	--------	---

Binary Accuracy: 100.00%



Conclusion:

- The 2-4-1 network with Tanh hidden and Sigmoid output layers is sufficient to solve XOR.
 - Gradient checking ensured that the backpropagation implementation is correct.
 - Training demonstrates the neural network library works for small, non-linear problems.
-